AI-BASED DRIVER SAFETY MONITORING AND INTELLIGENT INCIDENT RECORDING SYSTEM

Project Context & Implementation Guide for AI-Assisted Development

Final-Year Engineering Project • Information Technology • 8th Semester

Purpose of this document

This guide is intended to be attached to an AI coding assistant in VS Code and used as the source of truth while building the project. It records the product goals, approved stack, functional requirements, architecture, algorithms, data model, implementation sequence, testing plan, constraints, and working rules.

Core design principle

Build a free-first browser-based driving simulator and driver-monitoring prototype. Keep computer vision, event logic, video buffering and initial incident storage on-device. Cloud services are optional and must not be necessary for core functionality.

Version 1.0 • Prepared 20 October 2026

How to Use This Context Guide

- Give this PDF to the AI assistant before asking it to code. Ask it to read the entire document and summarize the requirements before implementation.
- Build in phases. Do not request the entire application in one prompt.
- Ask the assistant to inspect the current repository before changing files.
- Require Yarn commands only, a list of changed files, a short explanation, and test/verification steps after each phase.
- Treat this as a university prototype, not a certified automotive safety product.

Initial prompt for the AI coding assistant

```
You are the senior engineer assisting with this final-year IT project.
Read the attached Project Context & Implementation Guide fully before coding.
Treat it as the source of truth. First inspect the existing repository and report:
1. current project structure,
2. missing requirements,
3. a phased implementation plan,
4. technical risks and browser/device constraints.
Do not rewrite the whole project or install packages until you explain the plan.
Use Yarn commands only. Keep core features free and local-first.
Do not add paid APIs, paid services, secrets, or unrelated features.
Implement one phase at a time and provide tests and manual verification steps.
```

1. Project Overview

1.1 Project title

AI-Based Driver Safety Monitoring and Intelligent Incident Recording System Using Camera and Device Sensors

1.2 Description

A browser-based educational prototype simulating a driving session while monitoring a driver through an available camera. It estimates drowsiness from temporal eye-closure patterns, estimates distraction from head orientation, warns when simulated speed exceeds a configurable limit (initially 20 km/h), analyses supported device-motion signals for possible collision-like events, and preserves approximately 120 seconds of pre-event video when a critical event occurs. Incident metadata is stored locally; cloud synchronization is optional and out of scope for the first version.

1.3 Product boundaries

- The initial product is a browser-based simulator and experimental monitoring prototype.
- It does not control real vehicle brakes, steering, engine or other vehicle systems.
- It identifies potential incidents; it cannot guarantee that a collision occurred.
- Core features should work without login, internet access, paid API keys or cloud services.
- A browser cannot guarantee uninterrupted recording if the tab is suspended, the browser is closed, the device locks or storage runs out.

2. Final Technology Stack

Area	Selected stack	Purpose
Application	Next.js App Router + TypeScript	Pages, routing, application shell and UI.
UI and styling	Tailwind CSS + shadcn/ui	Responsive dashboard, controls, dialogs and status panels.
Driving simulation	Phaser 3	2D simulated road, vehicle, speed and repeatable events.
Computer vision	MediaPipe Tasks Vision (JavaScript)	Facial landmarks for eye and head-pose features.
Camera and recording	getUserMedia + MediaRecorder APIs	Camera permission, preview and short video segments.
Motion sensors	DeviceMotionEvent + DeviceOrientationEvent	Experimental movement/orientation signals where supported.
Local data	Dexie.js + IndexedDB	Incident metadata and video-segment blobs stored in the browser.
Speed source	Phaser simulation state; optional Geolocation API	Deterministic demo speed; optional real-world location estimate.
Package manager	Yarn	Dependency and script management.
Version control	Git + GitHub	History, collaboration and backup.
Hosting	Vercel Hobby	Deploy within the current plan's limits.

No backend framework, hosted database, AI API, cloud video storage or paid map service is required for version 1.0. If cloud sync is later approved, evaluate Supabase Free for authentication and structured records, but keep videos local by default.

2.1 Stack implementation rules

- Use compatible stable versions; do not blindly force package versions that conflict.
- Use Yarn commands and commit yarn.lock.
- Load MediaPipe and browser APIs only in client components. Never access camera/sensor APIs during server-side rendering.
- Phaser owns the simulation canvas; React/Next.js owns the surrounding interface.
- IndexedDB is local browser storage, not a server database. Users or browser policies can clear it, and storage is quota-limited.
- Free hosting tiers have limits and can change. Never promise unlimited free hosting.

3. Functional Requirements

ID	Requirement	Acceptance criteria
FR-01	Dashboard/session start	User can open the dashboard and start a test session.
FR-02	Permissions	Camera permission is requested after user action; denial is explained without crashing.
FR-03	Camera preview	Preview starts/stops on a supported HTTPS or localhost origin.
FR-04	Face landmarks	Show face-detected status. No face is not automatically classified as drowsiness.
FR-05	Drowsiness estimate	Use eye landmarks and duration; suppress alerts for ordinary brief blinks.
FR-06	Distraction estimate	Estimate head orientation; warn only after sustained deviation from calibrated forward range.
FR-07	Overspeed	Show simulation speed; speed above the configured threshold (default 20 km/h) triggers a warning and event.
FR-08	Motion analysis	Use supported motion/orientation signals; missing sensors or denied permission must not crash the app.
FR-09	Potential incident	Combine motion features and optionally a simulator-generated event; label as potential incident, not confirmed accident.
FR-10	Rolling buffer	Maintain about 120 seconds of segments and preserve them when a test incident occurs.
FR-11	Incident record	Save timestamp, event type, speed, driver state, sensor summary and video reference.
FR-12	Incident history	List incidents and open details or play the saved clip.
FR-13	Local-first	Core simulation, alerts and storage work without a cloud account.
FR-14	Privacy controls	Provide clear camera state, stop recording control and delete/export options where feasible.

4. Non-Functional Requirements

- Performance: keep UI responsive; process vision at a controlled frame rate instead of every available frame if performance suffers.
- Reliability: handle permission denial, missing sensors, camera disconnects, model-load failure, storage quota errors and malformed data.
- Usability: clear statuses, readable alerts, visible start/stop controls and a simple incident-history workflow.
- Privacy: process frames locally by default; do not upload or permanently store routine footage without explicit user action.
- Maintainability: TypeScript strict mode, small modules, documented thresholds, clear types and no large monolithic components.
- Portability: support current desktop browsers first; test mobile browsers separately because APIs and background behavior differ.
- Accessibility: do not rely on color alone; provide text labels for warning levels and keyboard-accessible controls.

5. Scope and Explicit Non-Goals

Included in version 1

- 2D driving simulator with controllable speed and repeatable events.
- Driver-facing camera preview and face-landmark processing.
- Experimental eye-closure/drowsiness and head-pose/distraction estimation.
- Overspeed alerts using simulated speed.
- Optional sensor-driven potential-incident scoring.
- Rolling pre-event video buffer and incident preservation.
- Local incident history, detail view, deletion and export where feasible.
- A dashboard for current state, events and simple session statistics.

Not included in version 1

- Certified real-world accident detection or medical diagnosis.
- Vehicle control, autonomous driving, lane keeping or braking.
- Automatic emergency service calls, SMS gateways or paid notifications.
- Cloud GPU inference, paid AI APIs, paid map APIs or cloud video upload.
- Complex 3D driving, traffic simulation, license-plate recognition or unrelated features.

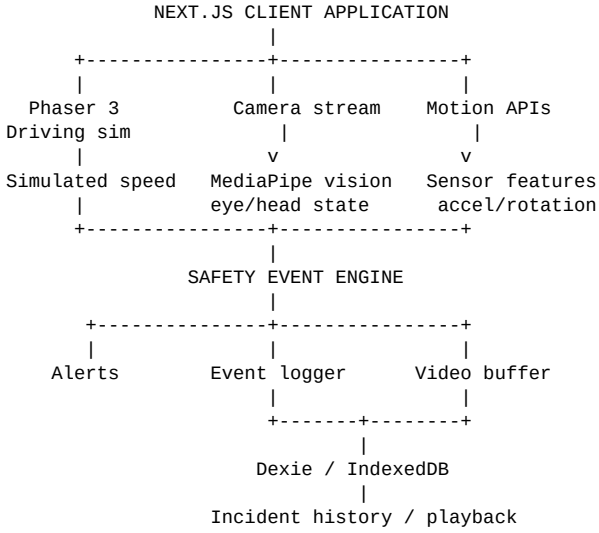
6. User Interface and Main Screens

Screen	Required content
Home / dashboard	Project status, start session, camera/sensor readiness, recent events and safety disclaimer.
Driving simulator	Road/canvas, simulated vehicle, speedometer, speed control, pause/end session, camera panel and warning panel.
Driver monitoring	Face detected/not detected, eye state, head-pose state, drowsiness/distraction status, confidence or quality indicator.
Incident history	Incident list with time, type, severity, speed and recording availability.
Incident detail	Event summary, pre-event clip playback, optional post-event clip, sensor summary and delete/export actions.
Settings	Speed threshold (default 20 km/h), recording duration (120 seconds), alert preferences and data-retention controls.

UI behavior

- Do not show a fabricated 'accurate' safety score before there is a defensible formula and validation. If a score is implemented, label it experimental and explain the components.
- Show a clear status for each subsystem: ready, running, permission needed, unsupported or error.
- Use a confirmation dialog before deleting a saved incident.
- Never play audio alerts at a volume that could startle users during actual driving; test only in a controlled environment.

7. High-Level Architecture



The browser-based system is a single client application for the first version. No server is required for detection or local persistence. Hosting serves the web application only; the actual session processing happens in the user's browser.

8. Recommended Folder Structure

```
driver-safety-system/
  app/
    layout.tsx
    page.tsx
    simulator/page.tsx
    incidents/page.tsx
    incidents/[id]/page.tsx
    settings/page.tsx
  components/
    dashboard/
    simulator/
    camera/
    monitoring/
    incidents/
    alerts/
    ui/
  features/
    vision/
      face-landmarker.ts
      eye-aspect-ratio.ts
      drowsiness-engine.ts
      head-pose-engine.ts
    sensors/
      motion-sensor.ts
      motion-features.ts
      incident-detector.ts
    recording/
      media-recorder.ts
      rolling-buffer.ts
      incident-preservation.ts
    simulation/
      phaser-game.ts
      simulation-state.ts
    events/
      event-bus.ts
      safety-engine.ts
  lib/
    db.ts
    schema.ts
    permissions.ts
    logger.ts
    config.ts
  types/
    driver.ts
    event.ts
    incident.ts
    recording.ts
  public/
    models/
  tests/
  docs/
    PROJECT_CONTEXT.md
  .env.example
  package.json
  yarn.lock
  README.md
```

This is a suggested structure, not a requirement to create every file at once. Keep modules small and create files when their phase begins.

9. Core Data Models

9.1 Incident record

```
type IncidentType =
  | "overspeed"
  | "drowsiness"
  | "distraction"
  | "potential_collision"
  | "manual_test";

type IncidentRecord = {
  id: string;                  // UUID
  sessionId: string;
  type: IncidentType;
  createdAt: string;           // ISO timestamp
  speedKmh?: number;
  severity: "info" | "warning" | "high";
  driverState?: "alert" | "drowsy" | "distracted" | "unknown";
  faceDetected?: boolean;
  sensorSummary?: {
    accelerationMagnitudePeak?: number;
    rotationRatePeak?: number;
    speedDeltaKmh?: number;
  };
  preEventRecordingId?: string;
  postEventRecordingId?: string;
  notes?: string;
};
```

9.2 Recording metadata

```
type RecordingRecord = {
  id: string;
  incidentId?: string;
  sessionId: string;
  kind: "rolling_segment" | "incident_pre_event" | "incident_post_event";
  startedAt: string;
  endedAt: string;
  mimeType: string;
  sizeBytes: number;
  blob?: Blob;                  // local IndexedDB only
  retained: boolean;
};
```

9.3 Driving session

```
type DrivingSession = {
  id: string;
  startedAt: string;
  endedAt?: string;
  mode: "simulation";
  speedLimitKmh: number;       // default 20
  status: "active" | "paused" | "ended";
  incidentCount: number;
};
```

9.4 Storage rules

- Use Dexie tables for sessions, incidents and recordings.
- Store video Blobs in IndexedDB, not as base64 strings in localStorage.
- Keep database schema versions and migrations explicit.
- Handle quota errors and provide a message explaining that recording could not be saved.
- Do not persist raw face frames or face embeddings; store derived status/events only unless a user explicitly saves an incident video.
- When a rolling segment is older than 120 seconds and not retained by an incident, delete it from the rolling store.

10. Algorithms and Decision Logic

10.1 Drowsiness estimation

- Load face-landmark model client-side and process camera frames at a controlled rate.
- Find the eye landmarks and calculate Eye Aspect Ratio (EAR) or another documented eye-openness measure.

- Calibrate a threshold per session/user if feasible; a single threshold may not generalize across faces, camera positions or lighting.
- Use time-based logic: a brief closure is a blink; sustained closure or a high proportion of closed-eye frames in a sliding window may indicate possible drowsiness.
- Require enough valid face/eye observations. If the face is missing or the landmarks are low quality, return 'unknown' rather than 'drowsy'.
- Apply cooldown/hysteresis so one event does not trigger repeated alerts every frame.

Conceptual EAR equation for six eye landmarks:

```
EAR = (distance(p2,p6) + distance(p3,p5))
      / (2 * distance(p1,p4))
```

This is an experimental indicator, not a medical measurement. Calibrate and test it; do not claim it detects sleep with certainty.

10.2 Distraction estimation

- Estimate head yaw/pitch from facial landmarks using a documented approach.
- Define a forward-facing range during calibration.
- Start a timer when orientation remains outside that range; reset after return.
- Trigger an alert only after a configurable sustained duration, and mark the result as an estimate.
- Do not infer distraction solely because the face is briefly not detected.

10.3 Speed monitoring

```
speedKmh = simulationState.speedKmh
limitKmh = settings.speedLimitKmh ?? 20

if (speedKmh > limitKmh) {
  emitEvent("overspeed", { speedKmh, limitKmh })
  showWarningWithCooldown()
}
```

Use simulation state as the authoritative speed in the first version. Browser GPS speed is optional and must not replace simulation speed in reproducible tests.

10.4 Potential collision detection

- Collect acceleration and rotation samples only when the relevant API is supported and permission has been granted.
- Extract features over a time window: peak acceleration magnitude, change in acceleration, rotational rate, orientation change and (if meaningful) speed change.
- Normalize features and use a configurable heuristic score for the prototype. Document that weights and thresholds are experimental.
- Avoid treating every shake, pothole-like event or phone movement as a collision.
- Allow the simulator to generate deterministic test events so the end-to-end flow can be demonstrated even if physical sensors are unavailable.
- Label the result 'potential collision' or 'abnormal motion event', never 'confirmed accident'.

```
score = wA * normalizedAcceleration
      + wG * normalizedRotation
      + wV * normalizedSpeedChange
      + wO * normalizedOrientationChange

if score >= highThreshold:
    emit potential_collision
elif score >= warningThreshold:
    emit abnormal_motion_warning
else:
    no incident
```

Do not use the example equation as a validated crash model. Select weights/thresholds from controlled tests and report false positives and false negatives.

11. Rolling Video Buffer — Critical Design

11.1 Requirement

The system must preserve approximately 120 seconds of video recorded before a potential incident. A configurable post-event period (initially 30 seconds) may be added after the core pre-event buffer is reliable.

11.2 Browser implementation

- Ask for camera permission after a clear user action; use getUserMedia.
- Create MediaRecorder with a supported MIME type selected through MediaRecorder.isTypeSupported; do not assume every browser supports the same codec.
- Record short chunks, for example 5–10 seconds each. Keep a queue that covers at least 120 seconds, with a small safety margin.
- Use timestamps for each segment. On a trigger, freeze/copy the segments that cover the preceding 120 seconds so the cleanup routine cannot delete them.
- Keep collecting post-event segments for the configured duration, then mark the selected segments as retained.
- Store retained Blob segments in IndexedDB and link their IDs to the incident.
- Provide playback of segments in order. Concatenating independently recorded WebM chunks into one file may not always work reliably; prefer segment-by-segment playback or test concatenation thoroughly.
- Delete old non-retained segments; never delete a segment linked to a saved incident unless the user deletes that incident.
- Handle camera stop, page navigation, tab suspension, device sleep, storage quota errors and recording errors.

11.3 Buffer logic

```
onDataAvailable(blob, startTime, endTime):
    save segment to temporary rolling storage
    remove unretained segments older than bufferWindow
    keep enough segments to cover >= 120 seconds

onPotentialIncident(event):
    select segments intersecting [event.time - 120s, event.time]
    mark them retained before cleanup can run
    create incident record
    record post-event segments for configured duration
    attach retained segment IDs to incident
    notify UI of success or failure
```

11.4 Important implementation limitation

Browser MediaRecorder chunking is not a frame-perfect dashcam ring buffer. The exact amount of pre-event footage depends on segment duration, browser behavior and timing. The application must display recording status and report the actual captured time range. Do not silently claim two full minutes if the stored segments do not cover two minutes.

12. Permissions, Security and Privacy

- Camera: request only when the user starts monitoring; show a clear active-camera indicator and stop button.
- Motion/orientation: detect support and request permission on platforms that require a user gesture.
- Geolocation: optional; request only if the user enables GPS mode.
- Use HTTPS in deployed environments. Localhost is generally treated as a secure context for development.
- Never put service-role keys, private API keys or secrets in client-side environment variables.
- Do not upload video or facial data to a server in version 1.
- Provide clear delete controls for saved incidents and explain where data is stored.
- Avoid collecting identity or biometric templates. Facial landmarks should be transient processing data, not user-identification data.
- Include a disclaimer that alerts may be wrong and the app must not be used as the sole safety system in a moving vehicle.

13. Error Handling and Edge Cases

Case	Expected behavior
Camera permission denied	Explain how to grant permission; keep simulation available without camera.
No face detected	Show 'face unavailable'; do not label as drowsy.
Low light / poor landmarks	Show low-confidence/limited monitoring state; avoid overconfident alerts.
Model load fails	Show a recoverable error and a retry action; other simulator features remain usable.
Motion sensor unsupported	Disable physical-sensor mode; keep simulator-generated test events available.
Motion permission denied	Explain limitation; do not repeatedly prompt.
MediaRecorder unsupported MIME type	Try another supported type; if none, disable recording with a clear message.
IndexedDB quota exceeded	Stop accepting new retained recordings safely, preserve metadata if possible, and notify user.
Tab/background suspension	Mark monitoring as potentially interrupted; do not claim continuous monitoring.
Duplicate event triggers	Use event cooldowns and idempotent incident creation.
Storage deletion by browser	Handle missing blobs in incident details without crashing.
Network unavailable	Core local functions continue; no cloud sync should be required.

14. Testing and Acceptance Plan

14.1 Unit tests

- EAR calculation handles valid landmarks, zero/near-zero denominator and invalid points.
- Drowsiness timer distinguishes short closures from sustained closures.
- Distraction timer resets correctly when head pose returns to forward range.
- Speed threshold tests include 19.9, 20.0 and 20.1 km/h; warning should trigger only when speed is greater than the threshold.
- Incident score tests cover normal, warning and high-score ranges.
- Rolling buffer selects segments that cover the previous 120 seconds and protects retained segments from cleanup.
- Database operations create, read, update and delete sessions, incidents and recordings.

14.2 Integration tests

- Start session → camera preview → face state → warning → event record.
- Increase simulated speed above 20 km/h → overspeed alert → incident appears in history.
- Trigger a simulated potential collision → rolling segments retained → incident detail plays video segments.
- Deny camera permission → simulator remains usable.
- Simulate storage error → UI reports failure rather than showing false success.

14.3 Manual acceptance checklist

- A user can start and end a session without a page refresh.
- The camera can be started and stopped, and the UI reflects its state.
- Face-not-detected, drowsiness and distraction states are visibly distinct.
- The speedometer follows simulation state and the threshold is configurable.
- A test incident preserves the available pre-event window and records the actual time span.
- Incident details remain available after reload when IndexedDB data remains present.
- User can delete an incident and its linked recordings.
- No paid service or secret is needed for the main demonstration.

14.4 Evaluation metrics for the report

- Drowsiness precision/recall on a small, consented, controlled test set; clearly describe dataset limitations.
- False warnings per test minute for drowsiness, distraction and overspeed.

- Potential incident detection rate and false-positive rate across scripted movement scenarios.
- Time from trigger to warning and time from trigger to completed incident preservation.
- Percentage of tests in which the stored recording covers the intended pre-event duration.
- Browser CPU/memory usage and storage size during a session.

15. Development Roadmap

Phase	Deliverables	Done when
0. Repository setup	Next.js + TypeScript, Tailwind, shadcn/ui, Yarn, lint/typecheck/test scripts, Git.	App runs locally and baseline checks pass.
1. UI shell	Dashboard, simulator route, incident route, settings and responsive layout.	Navigation works with placeholder data.
2. Simulator	Phaser canvas, speed state, start/pause/end, deterministic test controls.	Speed and scripted events can be reproduced.
3. Camera	Permission flow, live preview, start/stop and error states.	Camera lifecycle works on target browser.
4. Vision	MediaPipe face landmarks, eye-state estimate, head-pose estimate.	Live states update and missing-face cases work.
5. Safety engine	Speed rules, drowsiness/distraction temporal rules, cooldowns and event bus.	Rules create predictable alerts and events.
6. Local database	Dexie schema, sessions, incidents, recording metadata and migrations.	Records persist across reloads.
7. Video buffer	Short segments, 120-second queue, event retention, post-event capture and playback.	A test event preserves the pre-event video window.
8. Sensor module	Motion permission/support detection, features, heuristic score, simulation fallback.	Sensor and simulated events can trigger incident workflow.
9. Dashboard integration	Live statuses, event history, incident details, delete/export.	Complete demo can run from start to review.
10. Hardening	Unit/integration tests, performance, privacy, documentation and demo script.	Acceptance checklist passes; limitations are documented.

Do not start the next phase until the current phase has a working demo and its core tests pass.

16. Commands and Development Workflow

Use Yarn commands only. The AI assistant should inspect the current package manager and repository before suggesting commands. A typical initial setup for a new project is:

```
yarn create next-app driver-safety-system
cd driver-safety-system
yarn dev
```

When adding dependencies, the assistant should explain why each package is needed and use commands such as `yarn add ` or `yarn add -D `. Exact dependency names and installation steps should be checked against current official documentation before use.

Expected scripts

```
yarn dev        # local development server
yarn build      # production build
yarn start      # run production build locally
yarn lint       # lint if configured
yarn typecheck  # TypeScript validation if configured
yarn test       # tests if configured
```

Do not invent scripts that are not in package.json. Add scripts when setting up the repository and keep them accurate.

17. Git and AI Collaboration Rules

- Use small commits after each completed phase.
- Never overwrite a working codebase wholesale without first inspecting and explaining the plan.
- Do not silently remove existing functionality or change the approved stack.
- Do not install multiple competing libraries for the same job without a reason.

- Do not store API keys in source control. Keep .env.example free of real secrets.
- For each task, ask the AI to list files changed, decisions made, tests run, manual checks needed and known issues.
- If a browser API limitation is found, explain it and propose a minimal fallback before changing architecture.
- Prefer accessible UI, typed interfaces and small reusable modules over giant components.

18. Risks and Mitigations

Risk	Mitigation
Browser suspends recording	Keep the tab active for demo, show recording status and document that browser recording is not guaranteed in background.
Two minutes not preserved exactly	Use short timestamped segments with margin; verify actual coverage and display it.
False drowsiness alerts	Use temporal logic, calibration, confidence checks and cooldowns.
False incident alerts	Use sensor fusion heuristics plus deterministic simulation tests; call them potential incidents.
Poor camera conditions	Show degraded/unknown status and allow testing in good lighting.
Local storage limits	Estimate storage, delete unretained segments and handle quota errors.
Free hosting plan changes	Keep source portable and core features local; check current hosting terms.
Overly broad scope	Freeze version 1 scope and defer cloud sync, native Android and advanced AI.

19. Definition of Project Success

Version 1.0 is considered successful when a reviewer can start the web application, launch the driving simulator, enable the camera, observe driver-state estimates, exceed the 20 km/h simulated threshold to produce an alert, trigger a scripted potential incident, preserve the available two-minute pre-event recording, and review the event from local incident history. The application must handle denied permissions and missing sensor support without crashing. The report must document the testing procedure, observed results, false alerts and limitations.

20. Final Demo Scenario

```
1. Open dashboard and start a new simulation session.
2. Grant camera permission and show live preview.
3. Show face-detected status and available eye/head indicators.
4. Increase simulated speed to 21 km/h; show overspeed warning.
5. Trigger a controlled drowsiness/distraction test or explain limitations if
   a live test cannot reliably reproduce it.
6. Allow recording buffer to accumulate for at least 120 seconds.
7. Trigger the simulator's potential-collision test event.
8. Show incident preservation progress and completion/failure state.
9. Open incident history and review metadata plus saved video segments.
10. Demonstrate deletion and explain local privacy/storage behavior.
```

21. References and Official Documentation

Use official documentation as the source of truth for APIs and current package setup. Confirm the current version and browser support during implementation.

- Next.js documentation: https://nextjs.org/docs
- Tailwind CSS documentation: https://tailwindcss.com/docs
- shadcn/ui documentation: https://ui.shadcn.com/docs
- Phaser documentation: https://docs.phaser.io/
- MediaPipe Face Landmarker for Web: https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker/web_js
- MDN MediaDevices.getUserMedia: https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia
- MDN MediaRecorder: https://developer.mozilla.org/en-US/docs/Web/API/MediaRecorder
- MDN DeviceMotionEvent: https://developer.mozilla.org/en-US/docs/Web/API/DeviceMotionEvent
- MDN DeviceOrientationEvent: https://developer.mozilla.org/en-US/docs/Web/API/DeviceOrientationEvent

- MDN Geolocation API: https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API
- Dexie documentation: https://dexie.org/docs/
- IndexedDB API: https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API
- Vercel plans: https://vercel.com/docs/plans/hobby
- Supabase billing (optional future sync): https://supabase.com/docs/guides/platform/billing-on-supabase

22. Final Instruction to the AI Builder

Implement the smallest complete version first. Prioritize a reliable simulation, clear permissions, stable local storage and reproducible event recording over a large feature count. Do not claim drowsiness or collision detection is scientifically validated unless it has been evaluated with an appropriate dataset and methodology. Keep every feature demonstrable, explainable and testable. If any requirement conflicts with browser limitations, stop and explain the constraint before proposing a change.

End of project context guide.